Distributed Data: NoSQL and Beyond

Lecture 7 · Distributed Software Systems

CdLM Informatica – Università di Bologna

Learning outcomes

At the end of this lecture you should be able to:

1. **Distinguish** data models (how data is shaped and queried) from data distribution mechanisms (how data is placed and copied)
2. **Explain** partitioning, replication and the effect of network failures
3. **Reason** about CAP, ACID/BASE and consistency models for a concrete operation
4. **Compare** Cassandra, MongoDB, Redis, Neo4j, HBase and distributed SQL by the guarantees they offer

Roadmap

- **Part 1 · Data models**: key-value, document, wide-column, graph, relational
- **Part 2 · Data distribution**: scaling out, partitioning, replication, failures
- **Part 3 · Guarantees**: ACID, BASE, CAP, consistency models, CRDTs
- **Part 4 · Systems**: Cassandra, MongoDB, Redis, Neo4j, HBase, distributed SQL
- **Part 5 · Design practice**: the global shop, two exercises

Running thread: one application – a global shop – revisited in every part. Three short polls check understanding along the way.

A motivating scenario: a global shop

- Customers browse products from many regions; latency matters
- Inventory changes concurrently; orders must **not oversell**
- The service should survive server crashes and network failures
- Latency, availability, correctness and operational cost compete
- **One application can require different guarantees for different operations**

Question: if you had to relax one requirement, which one, and for which operation?

Database choice starts with workloads for the scenario

Operation	Access pattern	What matters most	Cost of a stale read
Catalog search	Read-heavy; flexible attributes	Secondary indexes, low latency	Low: outdated description
Shopping cart	Small keys; frequent updates	User-perceived latency	Medium: user does not see own update
Inventory	Concurrent decrements	Invariant: stock ≥ 0	High: overselling
Orders and payments	Multi-item writes, audit	Atomicity, durability	Very high: lost or duplicated money

Many data models, different query strengths

- **Key-value**: opaque value addressed by a key; get/put by key
- **Document**: nested, self-describing records (JSON-like); queries on fields
- **Wide-column**: rows grouped by partition key, sorted by clustering columns
- **Graph**: nodes and typed relationships; multi-hop traversals
- **Relational**: tables, joins, constraints, transactions, declarative SQL

A data model fixes which queries are natural – *not* how data is distributed or how consistent it is.

Key-value



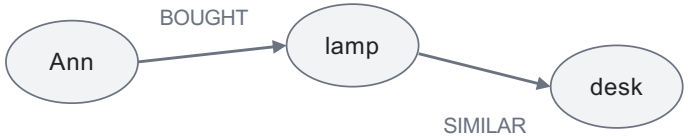
Document

```
{ id: 12, name: "lamp",
  attrs: { watt: 9, color: "red" } }
```

Wide-column



Graph



The three Vs: a historical motivation

- **Volume**: data size grows from terabytes to petabytes and beyond
- **Velocity**: data arrives continuously; batch processing gives way to streaming
- **Variety**: data shapes range from tables to JSON documents to graphs

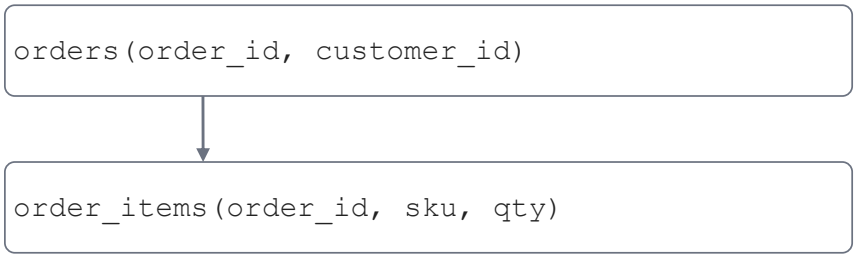
These are **workload dimensions**. They motivated the first NoSQL systems, but they do not determine the consistency guarantees a system offers.

Relational and document models

- **Relational**: entities in separate normalized tables, combined by joins
- **Document**: the whole aggregate (order with its items) stored together
- Embedding makes reading the aggregate cheap and keeps it on one node
- Normalization avoids duplication and makes cross-aggregate queries natural

Choose by access pattern: which data is read and updated together?

RELATIONAL



DOCUMENT

```
{
  order_id: 42,
  customer: "C7",
  items: [ {sku: "A", qty: 2},
           {sku: "B", qty: 1} ]
}
```

What NoSQL does – and does not – mean

- A broad family of **non-relational** data-management systems
- Many *do* use schemas, types, indexes, query languages and transactions
- Many are designed for horizontal scaling and replication – but so are distributed SQL systems
- “Not Only SQL” is a historical label, *not* an architecture specification

Common myths: NoSQL = schemaless · NoSQL = no transactions · NoSQL = eventually consistent. Each one has well-known counterexamples.

Check your understanding (scenario)

Orders are stored as documents that embed their items. Marketing now needs the total quantity sold per SKU across all orders. What is the most accurate statement?

A Embedding makes this query faster than in a relational schema

B The data must be migrated to a graph database

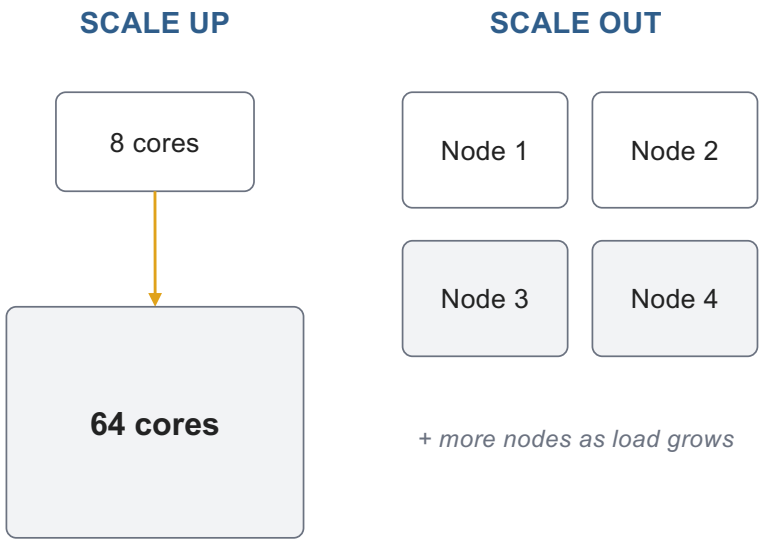
C It is possible, but needs an aggregation over many documents or a separate index/model

D It cannot be done with a document database

Vote first, then discuss with your neighbour and vote again

From one machine to many

- **Scale up** (vertical): more CPU, RAM or storage on one server
  - simple data placement; no distributed coordination
  - limited by machine size and cost; one point of failure
- **Scale out** (horizontal): add more machines
  - incremental growth, redundancy
  - data placement, coordination and **partial failures** become your problem



Partitioning: divide the key space

- Each record belongs to exactly **one partition** (shard), stored on one node group
- **Hash partitioning**: hash(key) decides the shard
  - even spread of load; range queries touch all shards
- **Range partitioning**: contiguous key intervals per shard
  - range scans stay local; risk of hotspots on recent keys
- Consistent hashing limits data movement when nodes join or leave



range partitioning of a numeric key

How to choose a partition key

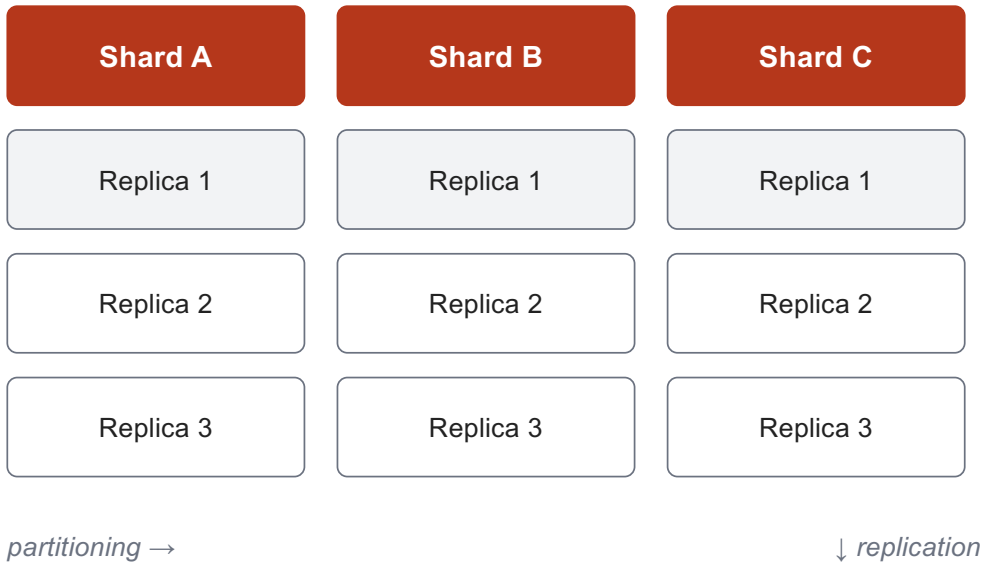
- Prioritize the **dominant reads and writes**
- Avoid hotspots: monotonic IDs, timestamps, “celebrity” keys
- Estimate cardinality, skew and future growth
- Preserve locality: which records must be read or updated together?

A partition key is an **architectural decision**, not mere syntax: changing it later means moving all the data.

Your turn: propose a partition key for the inventory of the global shop.

Partitioning and replication: two independent dimensions

- **Partitioning** distributes *different* keys across shards → capacity
- **Replication** keeps *copies* of each shard on several nodes → fault tolerance and read capacity
- The two are orthogonal: N shards × R replicas
- Replicas must be kept consistent – the subject of Part 3



Leader-based vs leaderless replication

Leader-based

- A leader orders all writes; followers replicate its log
- Failover needs leader election and **fencing** of the old leader
- Examples: MongoDB replica sets, Raft-based systems

Leaderless (Dynamo-style)

- Clients write to and read from several replicas
- Read/write **quorums**, read repair and anti-entropy reconcile values
- Examples: Cassandra, Riak

Recall Lecture 6: a replicated log is a totally ordered multicast.

Why failures change the semantics

- **Process crash**: a replica stops responding
- **Network partition**: live nodes cannot communicate reliably
- **Delayed message**: a stale value may still be served
- **Concurrent updates**: different replicas accept conflicting changes
- **Recovery**: what must be replayed, repaired or reconciled?

A slow node and a crashed node look the same from outside: **timeouts cannot tell them apart**.

Check your understanding (scenario)

Order IDs are assigned by an increasing counter and the orders table is range-partitioned on order_id into 8 shards. During a flash sale, what happens to new writes?

A They all go to the shard holding the highest IDs, creating a hotspot

B They are rejected until the shards are rebalanced

C They are spread evenly across the 8 shards

D They are replicated to all 8 shards

Vote first, then discuss with your neighbour and vote again

ACID is about transactions

- **Atomicity**: all-or-nothing execution of a transaction
- **Consistency**: the transaction preserves defined application invariants
- **Isolation**: concurrent transactions are controlled (e.g. serializable, snapshot)
- **Durability**: committed changes survive specified failures

ACID is compatible with distributed systems – but distributed transactions need coordination (2PC, consensus), which costs latency.

ACID “consistency” is not the C in CAP.

BASE and eventual convergence

- **Basically Available**: respond despite selected failures
- **Soft state**: replica state may change without new client operations (background propagation, repair)
- **Eventual consistency**: if updates stop and repair succeeds, replicas converge

BASE is an informal design philosophy, *not* the logical negation of ACID.

Eventual consistency says nothing about *what* a read returns before convergence.

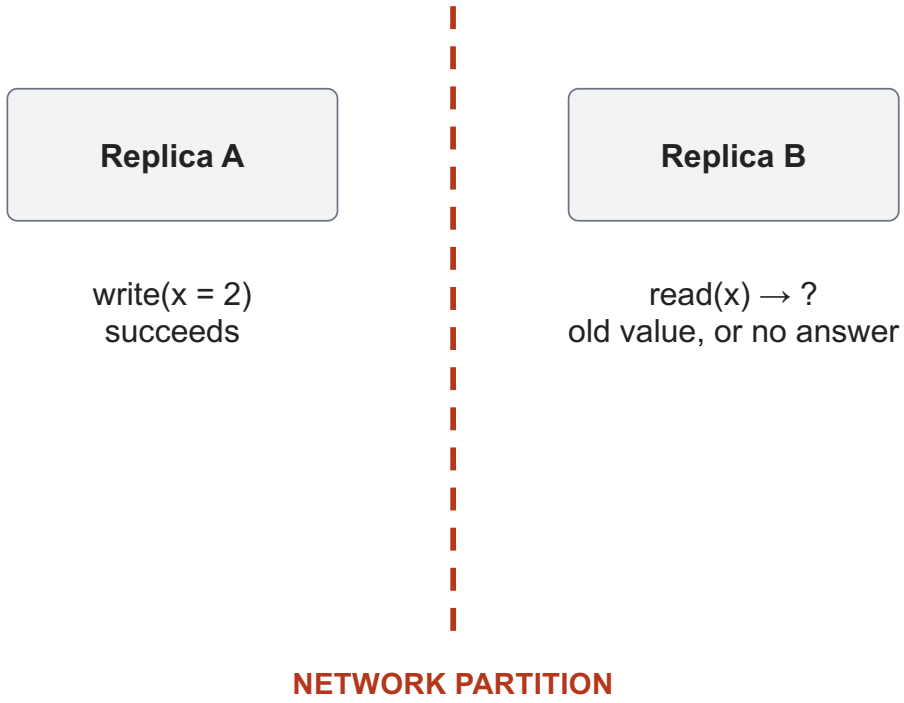
CAP: the precise trade-off

During a **network partition**, a replicated system must choose for each request:

- **answer** with possibly stale data → not linearizable
- **refuse or wait** → not available

C = linearizability · **A** = every non-failed node answers · **P** = messages between node groups may be lost

Gilbert and Lynch, 2002



CAP does not say “pick two forever”

- Outside partitions, systems can offer **both** strong consistency and availability
- During partitions, mechanisms may reject, delay or weaken *specific* operations
- Real services mix choices by operation, data and failure scope
- Latency matters even without partitions: **PACELC** – if Partition, A or C; Else, Latency or Consistency

CAP is an impossibility result, not a design method (Brewer, “CAP twelve years later”, 2012).

In-class exercise: what can you guarantee?

A network partition separates Europe from America for ten minutes. Inventory replicas: 3 in Europe, 2 in America.

1. Can a customer in America browse the catalog? With what data?
2. Can they add items to their cart? How are carts reconciled later?
3. Can they buy the last unit of a product? What do you return?
4. Which side can keep accepting orders, and why?

Pairs · 5 minutes · write one sentence of justification per answer.

Consistency is an application contract

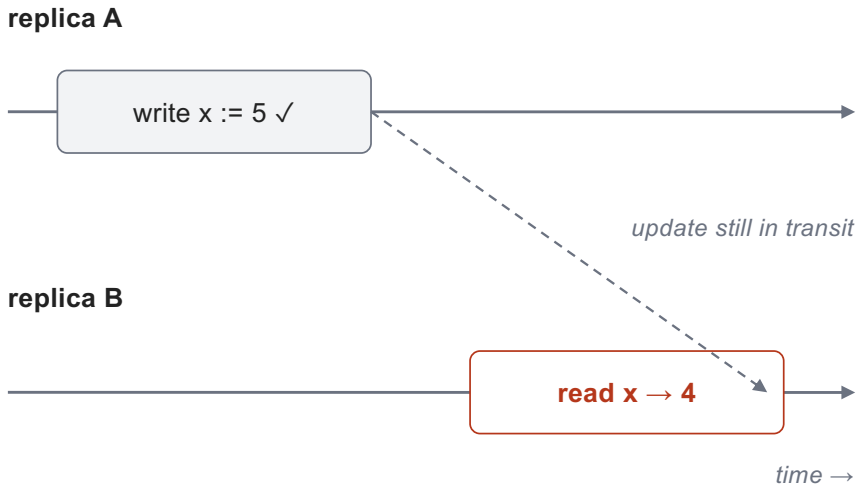
- **Linearizable**: a completed write is visible to every later read; operations appear atomic in real-time order
- **Sequential**: one global order, consistent with each client's program order
- **Causal**: writes that may influence each other are seen in that order by everyone
- **Eventual**: replicas converge when updates stop
- **Session guarantees**: read-your-writes, monotonic reads – per-client promises

Lecture 6 link: FIFO, causal and total-order multicast are the same ideas applied to messages.

A simple read anomaly

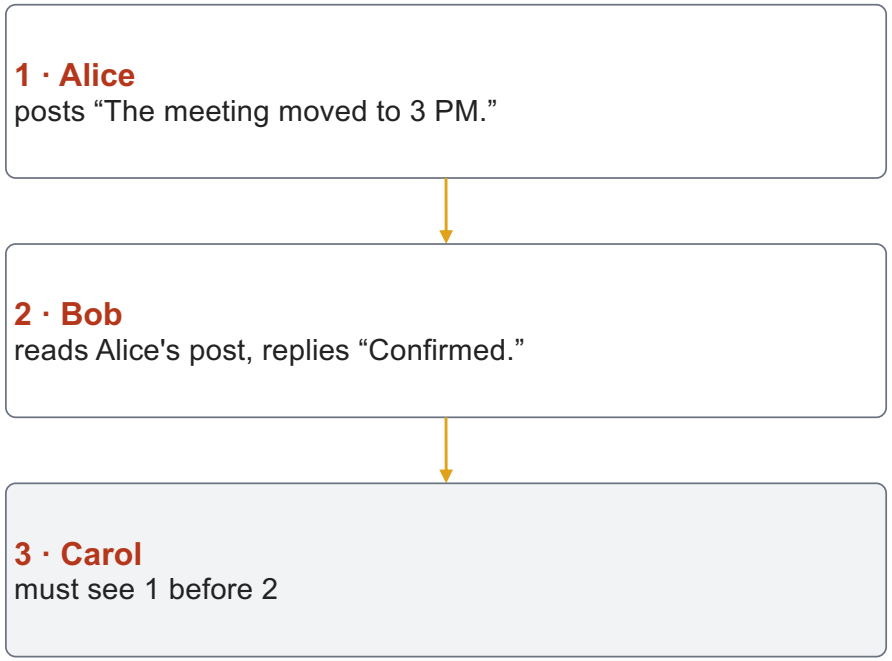
- Client 1 writes x := 5 at replica A; the write **completes**
- *Later*, client 2 reads x at replica B and gets 4: propagation lags
- Allowed under eventual consistency; **forbidden** under linearizability
- If client 1 itself reads 4, read-your-writes is violated

Always specify the order of write completion and read invocation.



Causal consistency: order matters

- Bob's reply **depends on** Alice's post: it must never be seen without it
- Causal order is a *partial* order: unrelated concurrent writes need not be globally ordered
- Implemented with dependency tracking (e.g. vector clocks)
- Causal consistency remains achievable while staying available under partitions



Consistency is not a single ladder

- Different guarantees constrain different observables
- Linearizability is stronger than sequential consistency: it adds **real-time order**
- Causal consistency (order) and convergence (eventual) address different concerns
- Read-your-writes and monotonic reads are **session** guarantees, layered per client

Design rule: choose the weakest guarantees that still preserve correctness.

CRDTs: convergence by construction

- **CRDT** = Conflict-free Replicated Data Type: concurrent updates merge deterministically
- **G-counter**: each replica increments its own entry; merge = element-wise maximum; value = sum
- Merge is commutative, associative, idempotent → no lost increments, no coordination
- Other types: PN-counter, OR-set (shopping carts), LWW-register
- **Limitation**: global invariants (stock ≥ 0) need coordination or escrow techniques

A CRDT is a data type, not a consistency level.

Check your understanding

During a network partition, a replica on the minority side keeps answering reads with the last value it knows. Which statement is correct?

A CAP does not apply because reads do not modify data

B The system is still linearizable because the value was once correct

C The system has violated ACID consistency

D The system has chosen availability over linearizability for those reads

Vote first, then discuss with your neighbour and vote again

Cassandra: Dynamo-inspired data distribution

- The **partition key** is hashed to a token on a ring; the token determines the replica nodes
- The **replication factor** (RF) defines how many copies each partition has
- Reads and writes choose a **consistency level** per request
- Hinted handoff, read repair and anti-entropy repair help replicas converge

Cassandra is a distributed wide-column store with an explicit CQL schema.

Cassandra: tunable consistency

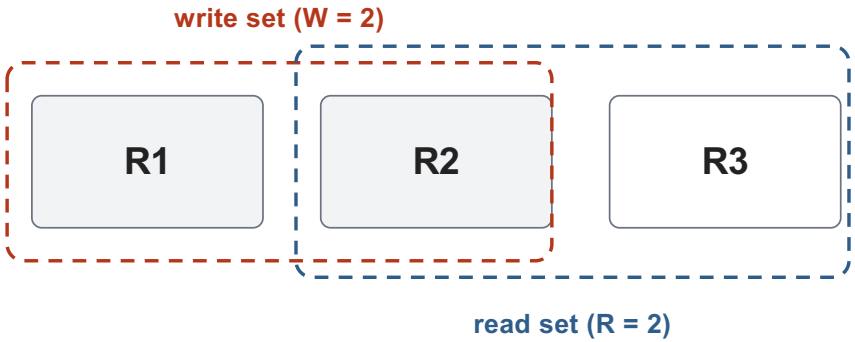
- **ONE**: one replica responds – low latency, weakest freshness
- **QUORUM**: a majority of the RF replicas responds – read and write sets overlap
- **ALL**: every replica responds – strongest requirement, least tolerant of outages
- Multi-datacentre variants: LOCAL_QUORUM, EACH_QUORUM

Quorum reasoning: if **R + W > N**, every read set intersects every write set.

Quorum example: RF = 3

- N = 3 replicas; write with QUORUM: **W = 2**; read with QUORUM: **R = 2**
- R + W = 4 > N = 3 → every read set **overlaps** the write set
- Intersection is necessary reasoning, *not* by itself a proof of linearizability

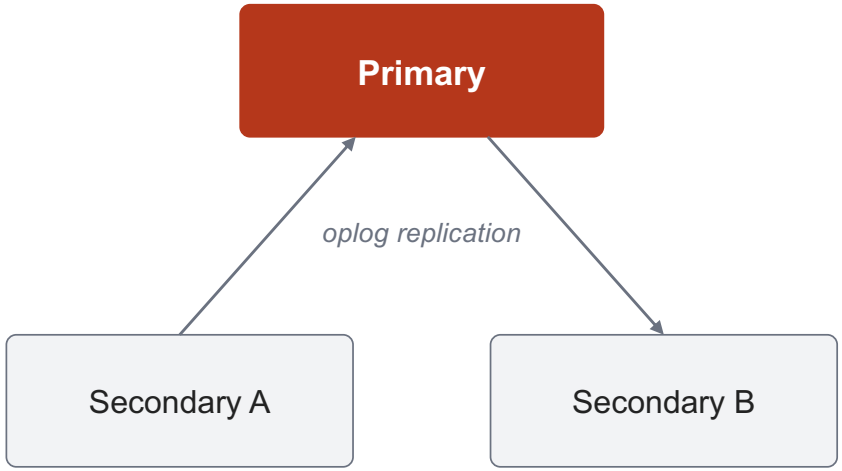
Question: what changes if one replica is unavailable?



R + W = 4 > N = 3

MongoDB: documents plus replica sets

- A **replica set**: one primary accepts writes; secondaries replicate its oplog
- If the primary fails, an **election** chooses a new one
- Sharded clusters partition collections by a shard key; each shard is a replica set
- Single-document updates are atomic; multi-document ACID transactions are also available



election after primary failure

MongoDB: read and write guarantees

- **Write concern**: how many members must acknowledge a write (e.g. w: "majority")
- **Read concern**: which data a read may return – local, majority, snapshot, linearizable
- **Read preference**: which members may serve the read – primary, secondary, nearest

Linearizable reads have conditions and costs; pair them with majority writes for the intended guarantee.

Causally consistent sessions give read-your-writes and monotonic reads.

Redis: latency and key-oriented access

- In-memory access to keys and rich data structures (lists, sets, hashes, streams)
- Useful for caching, sessions, counters and fast ephemeral state
- Replication is **asynchronous** by default: a failover can lose recently acknowledged writes
- Persistence (snapshots, append-only log) is an explicit design choice

A cache is not automatically the **source of truth**. Who invalidates stale entries?

Neo4j: relationships as first-class data

- **Property graph**: nodes, typed relationships, properties, traversal queries (Cypher)
- **Suitable workloads**: multi-hop questions – recommendations, dependencies, fraud rings, network paths
- Partitioning a graph is hard: traversals cross partition boundaries

Distribution and consistency are separate from the choice of the graph model.

HBase: one architectural insight to retain

- **Write path**: RegionServer → Write-Ahead Log → MemStore → HFiles
- **Why it matters**: durable logging, memory buffering, immutable sorted files and compaction – the **log-structured** design shared by Cassandra, RocksDB and others
- Strong consistency per row: one RegionServer serves a region at a time
- Cross-cluster replication is normally asynchronous

Internal implementation details are optional enrichment.

Distributed SQL: the false dichotomy

- **What remains relational**: data model, SQL, ACID transactions
- **What becomes distributed**: partitioned storage, replicated with consensus (Paxos, Raft), multi-node query execution
- Consensus gives strong guarantees but costs latency, especially across regions
- Examples: Google Spanner (TrueTime), CockroachDB, YugabyteDB

Relational vs NoSQL and centralized vs distributed are **distinct dimensions**.

Technology comparison: ask better questions

System	Dominant model	Replication	Architectural teaching point
Cassandra	Wide-column	Leaderless	Replication factor and per-request consistency level
MongoDB	Document	Leader-based (replica set)	Write concern, read concern, read preference
Redis	Key-value, in memory	Leader-based, asynchronous	Cache vs source of truth; acknowledged writes can be lost
Neo4j	Property graph	Leader-based cluster	Traversal locality; graphs are hard to partition
HBase	Wide-column on HDFS	Storage-level (HDFS)	Log-structured write path; strong per-row consistency
Spanner / CockroachDB	Relational	Consensus per partition	Distributed ACID transactions at a latency cost

Design case: e-commerce data ownership

Data	Possible store	Key guarantee	Acceptable weakness
Catalog	Document store / search index + cache	Flexible attributes, indexed reads	Stale reads for seconds
Cart	Key-value or document; CRDT set	Read-your-writes per session	Merging concurrent edits
Inventory	Transactional store or conditional updates	Never oversell (stock ≥ 0)	Refuse sales on the minority side
Orders and payments	Transactional (distributed SQL)	Atomicity and durability	Higher write latency

One possible design: compare it with your worksheet.

Exercise 1: partition and quorum

N = 3 replicas; a write with QUORUM succeeds. Then one replica becomes unreachable.

1. Can a QUORUM read complete? Which replicas may respond?
2. What can happen with ONE reads after a write that reached only one replica?
3. Is R + W > N sufficient to claim linearizability? Explain.

Pairs · 7 minutes · justify both safety (which values) and availability (does it complete).

Exercise 2: choose the data guarantees

For the global shop:

1. Model products, carts, inventory and orders
2. Pick partition keys and replication strategies
3. Specify read/write guarantees and transaction boundaries
4. Describe the behaviour of each operation during one network partition
5. Identify **one invariant** and **one acceptable stale read**

Teams of 3 · 10 minutes · submit one architectural decision and its trade-off.

Takeaways

- Data model, distribution mechanism and guarantees are **independent dimensions**
- Partitioning scales capacity; replication improves resilience but creates coordination needs
- Under partitions, each operation must choose between answering and being linearizable
- Consistency must follow application **invariants** and failure assumptions
- Choose the weakest guarantees that still preserve correctness

One-minute paper: the most important idea of today, and what is still unclear.

Selected references

M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly, 2017 – chapters 5–9

E. Brewer, “CAP twelve years later: how the ‘rules’ have changed”, *IEEE Computer*, 2012

S. Gilbert, N. Lynch, “Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services”, *SIGACT News*, 2002

D. Abadi, “Consistency tradeoffs in modern distributed database system design”, *IEEE Computer*, 2012

M. Herlihy, J. Wing, “Linearizability: a correctness condition for concurrent objects”, *ACM TOPLAS*, 1990

G. DeCandia et al., “Dynamo: Amazon's highly available key-value store”, *SOSP*, 2007

J. Corbett et al., “Spanner: Google's globally-distributed database”, *OSDI*, 2012

M. Shapiro et al., “Conflict-free replicated data types”, *SSS*, 2011